



南開大學
Nankai University

深度学习及应用实验报告

循环神经网络实验

课程名称：深度学习及应用

学院：密码与网络空间安全学院

姓名：金莫迪

学号：2312578

日期：2026 年 6 月 29 日

目录

1	实验目的	2
2	数据集与实验配置	2
3	模型原理	3
3.1	原始普通 RNN 模型结构	3
3.2	LSTM 模型结构	4
4	实验过程	5
5	实验结果与可视化	5
6	为什么 LSTM 性能优于普通 RNN	6
7	问题与改进方向	7
8	实验结论	7
	参考文献	8

表 1: RNN 实验训练配置

项目	设置
数据集	PyTorch tutorial names, 全量 20074 条样本
类别数	18 种语言: Arabic、Chinese、English、French、Russian 等
训练/验证划分	16059 / 4015
输入粒度	字符级序列
字符表大小	59
Embedding 维度	64
隐藏层维度	128
模型	单层 RNN、单层 LSTM
优化器	AdamW, 学习率 0.001, weight decay 0.0001
最大轮次	60 epoch
早停机制	监控验证 loss, patience=8
运行设备	CUDA

1 实验目的

本实验使用字符级循环神经网络完成姓名语言类别识别任务，目标是理解 RNN 对变长序列的建模方式，并比较普通 RNN 与 LSTM 在同一数据集和同一训练协议下的表现。普通循环网络可追溯到 Elman 对时间结构建模的研究 [1]，LSTM 则通过门控细胞状态缓解长序列训练中的梯度衰减问题 [2]。本实验要求输出模型结构、训练曲线、准确率曲线、混淆矩阵，并解释 LSTM 相比普通 RNN 的优势。

2 数据集与实验配置

数据来自 PyTorch 官方字符级姓名分类教程所使用的 `data/names` 数据集 [4]。每个文本文件对应一个语言类别，每一行是一个姓名。实验保留全部 18 个类别，按 80%/20% 划分训练集和验证集，共得到 16059 个训练样本和 4015 个验证样本。

预处理阶段将姓名统一转换为字符序列，并根据全体训练数据构建字符表。每个字符映射为整数索引，再通过 Embedding 层得到连续向量表示。由于姓名长度不同，batch 内使用 padding 对齐，并记录真实长度，避免模型把补齐字符当作有效输入。

表 1 中 RNN 与 LSTM 使用完全相同的输入表示、隐藏维度、优化器和数据划分，因此最终性能差异主要由循环单元结构决定。

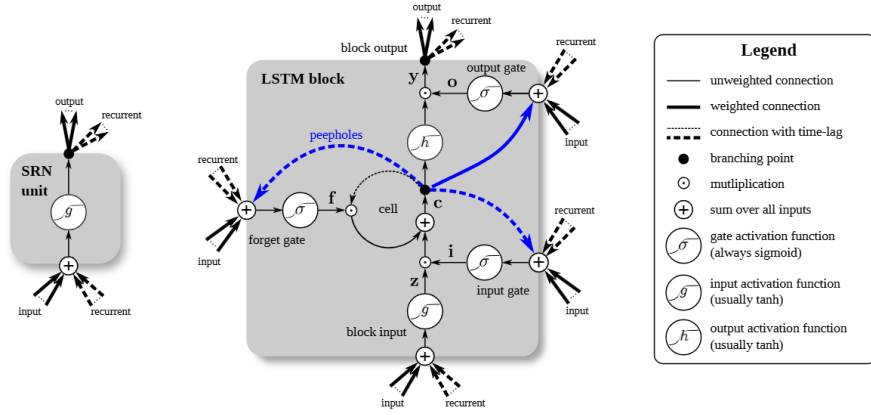


图 1: 简单循环单元与 LSTM block 结构对比 [3]

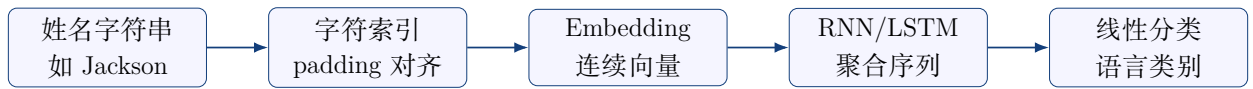


图 2: 字符级姓名语言识别流程

3 模型原理

字符级 RNN 每次读入一个字符向量 $\mathbf{x}_t$ ，并将当前输入与上一时刻隐藏状态融合：

$$\mathbf{h}_t = \tanh(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b}). \quad (1)$$

在姓名分类中，最后一个有效字符对应的隐藏状态被送入线性层，得到 18 个语言类别的 logits：

$$\mathbf{o} = \mathbf{W}_o \mathbf{h}_T + \mathbf{b}_o. \quad (2)$$

普通 RNN 的问题在于，式 (1) 会反复对历史信息进行非线性压缩。当序列较长或判别线索分散在不同位置时，早期信息容易被后续字符覆盖。

LSTM 在隐藏状态外引入细胞状态 $\mathbf{c}_t$ ，并使用遗忘门、输入门和输出门控制信息流：

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f), \quad \mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i), \quad (3)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c), \quad \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t. \quad (4)$$

式 (4) 中的加法路径使长期信息更容易跨时间步传播，这也是 LSTM 在序列任务中通常优于普通 RNN 的重要原因。图 1 展示了论文中的 SRN 与 LSTM 结构对比。

图 2 展示了本实验的端到端处理流程。模型不依赖人工语言规则，而是直接从字符组合中学习类别模式。

3.1 原始普通 RNN 模型结构

普通 RNN 由 Embedding 层、循环层和线性分类层三部分组成。Embedding 层将离散字符编号映射为 64 维连续向量，使模型能够学习字符之间的分布式表示；循环层逐字符

更新隐藏状态，隐藏层维度为 128；线性层将最后一个有效时间步的隐藏状态映射到 18 个语言类别。对于一个 batch，输入张量形状为 $B \times T$ ，其中 B 是 batch size， T 是当前 batch 中最长姓名长度。经过 Embedding 后变为 $B \times T \times 64$ ，循环层输出最终状态 $\mathbf{h}_T \in \mathbb{R}^{128}$ ，再由全连接层输出 $\mathbb{R}^{18}$ 的 logits。

普通 RNN 的优势是结构清晰、参数量较少，对短序列能够快速学习。但它的隐藏状态每一步都会被整体刷新，所有历史信息都压缩在同一个向量中。当姓名较长或语言线索位于中间位置时，早期线索容易被后续字符覆盖，因此普通 RNN 更容易出现记忆不足和训练波动。

3.2 LSTM 模型结构

LSTM 与普通 RNN 的输入输出接口相同，但循环单元内部结构更复杂。它同时维护隐藏状态 $\mathbf{h}_t$ 和细胞状态 $\mathbf{c}_t$ ，其中细胞状态更像长期记忆通道，隐藏状态则承担当前时间步输出。遗忘门控制旧记忆保留比例，输入门控制新信息写入比例，输出门控制当前分类可见的信息。与普通 RNN 相比，LSTM 把“保存长期信息”和“产生当前输出”分开，因此更适合处理判别线索位置不固定的姓名序列。

本实验中的 LSTM 使用单层结构，Embedding 维度为 64，隐藏层维度为 128，最终分类层仍输出 18 类 logits。为了公平比较，LSTM 与 RNN 使用完全相同的字符表、batch 构造方式、训练集划分、优化器和早停规则。二者主要区别只在循环单元内部的信息更新机制。

Listing 1: 原始 RNN 与个人实现 LSTM 网络结构打印结果

```
1  ===== RNN =====
2  CharSequenceClassifier(
3      (embedding): Embedding(59, 64, padding_idx=0)
4      (rnn): RNN(64, 128, batch_first=True)
5      (fc): Linear(in_features=128, out_features=18, bias=True)
6  )
7
8  ===== LSTM =====
9  CharSequenceClassifier(
10     (embedding): Embedding(59, 64, padding_idx=0)
11     (rnn): LSTM(64, 128, batch_first=True)
12     (fc): Linear(in_features=128, out_features=18, bias=True)
13 )
```

代码清单 1 给出了两个模型的实际打印结构。可以看到，二者共享 Embedding 层和最终全连接分类层，核心差异集中在循环层：普通 RNN 使用 `RNN(64,128)`，LSTM 使用 `LSTM(64,128)`。这保证了实验对比主要反映循环单元机制差异，而不是输入表示或分类器容量差异。

表 2 展示了两个模型的结构差异。由于姓名识别任务既依赖局部字符组合，也依赖前缀、后缀和中间字符模式，LSTM 的门控结构理论上更容易保留有效线索。

表 2: RNN 与 LSTM 模型结构对比

模型	状态形式	本实验结构	特点
RNN	单一隐藏状态 $\mathbf{h}_t$	Embedding(59,64) RNN(64,128) + Lin- ear(128,18)	+ 简洁快速, 但长期记忆能力有限
LSTM	隐藏状态 $\mathbf{h}_t$ 与细胞状态 $\mathbf{c}_t$	Embedding(59,64) LSTM(64,128) + Lin- ear(128,18)	+ 门控记忆更稳定, 适合较复杂序列依赖

表 3: 姓名语言识别验证结果汇总

模型	最优验证损失	验证准确率 (%)	早停 epoch
RNN	0.5912	81.97	21
LSTM	0.5462	83.44	18

4 实验过程

训练时, 每个 batch 包含字符索引矩阵、真实序列长度和语言标签。模型前向传播后使用交叉熵损失进行监督学习, 并在每个 epoch 后计算验证集 loss 和 accuracy。实验采用早停策略: 若验证 loss 连续 8 次没有改善, 则停止训练并记录最佳验证 loss 对应的模型表现。RNN 在第 21 个 epoch 触发早停, LSTM 在第 18 个 epoch 触发早停。

为了分析模型错误模式, 实验在验证集上统计预测矩阵, 即按真实类别与预测类别组成的混淆矩阵。由于 names 数据集类别数量差异较大, 直接绘制原始计数会使 Russian、English、Arabic 等大类主导色阶, 小类别即使预测正确也难以观察。因此本文采用逐行归一化混淆矩阵, 每一行表示某个真实类别内部的预测比例。实验同时绘制 loss 曲线、accuracy 曲线、最终准确率柱状图和类别级预测矩阵, 从总体性能与类别级误判两个角度评价模型。

5 实验结果与可视化

表 3 显示, LSTM 的验证准确率达到 83.44%, 高于普通 RNN 的 81.97%, 验证损失也更低。虽然两者差距不大, 但在同一训练配置下, LSTM 体现出更好的序列记忆能力。

图 3 表明, 两个模型前期都能快速学习姓名字符模式; LSTM 在较少 epoch 内达到更低验证 loss, 而普通 RNN 后期波动更明显。结合表 3 可知, LSTM 的改进来自更稳定的验证性能, 而不是单纯训练更久。

图 4 显示 LSTM 的最终准确率更高。对于字符级姓名识别任务, 1-2 个百分点的提升说明门控结构对复杂字符组合具有实际帮助。

图 5 展示了类别级预测分布。逐行归一化后, 每个格子的颜色表示该真实类别被预

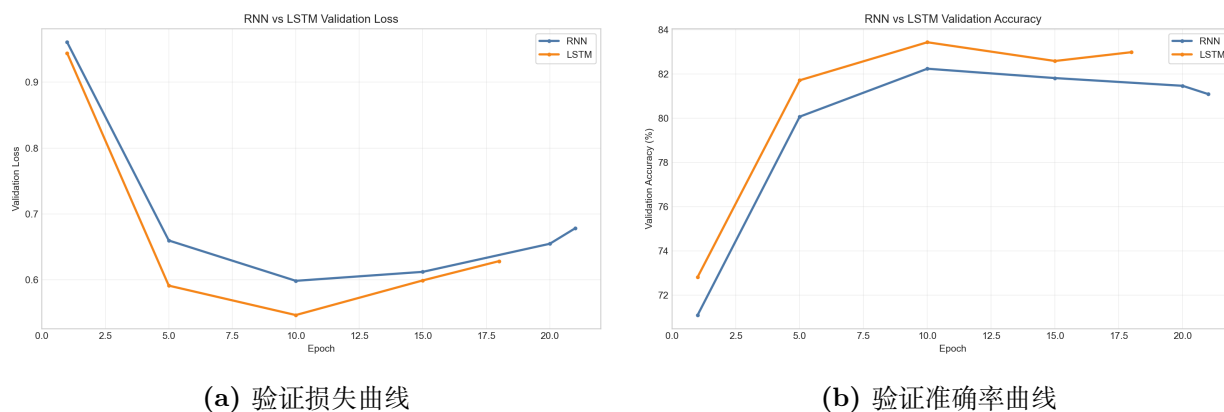


图 3: RNN 与 LSTM 训练过程对比

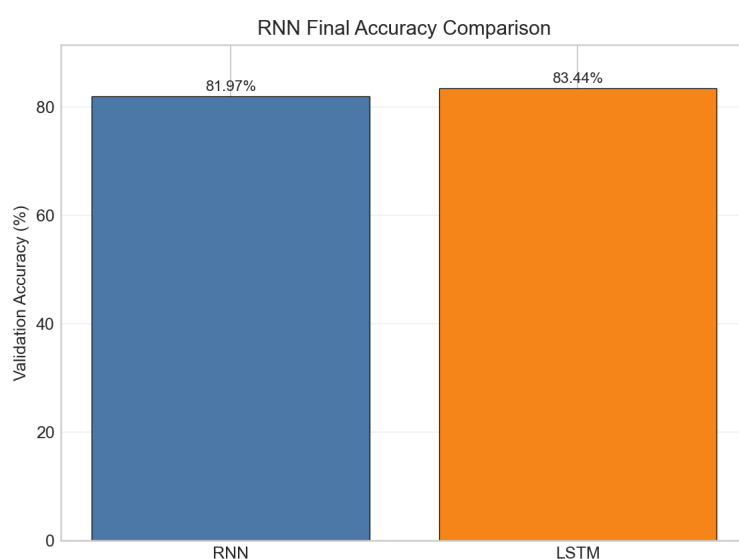


图 4: RNN 与 LSTM 最终验证准确率对比

测为某一类别的比例，因此可以排除类别样本数差异带来的视觉偏差。LSTM 在 Chinese、English、Greek、Irish、Japanese、Polish、Spanish 等类别上的对角线比例高于普通 RNN，说明其在多类姓名模式上具有更稳定的识别能力；但 Scottish、Portuguese、Vietnamese 等样本较少且拼写模式容易与 English、Chinese 或 Spanish 混淆的类别仍然表现较弱。

6 为什么 LSTM 性能优于普通 RNN

姓名语言识别依赖字符局部模式和跨位置组合。例如某些语言的后缀更具判别性，某些语言的前缀或中间字母组合更重要。普通 RNN 将历史信息压缩到单一隐藏状态中，后续字符会持续刷新状态；LSTM 则通过细胞状态保留更稳定的信息，并利用门控机制选择性更新。因此，在图 3 中，LSTM 更快达到较优验证 loss，并在表 3 中取得更高准确率。

从图 5 看，模型的错误主要集中在字符形式相近或样本量较少的语言类别之间。例如 English、Irish、Scottish 之间共享大量罗马字母姓名模式，French、Portuguese、Spanish 也

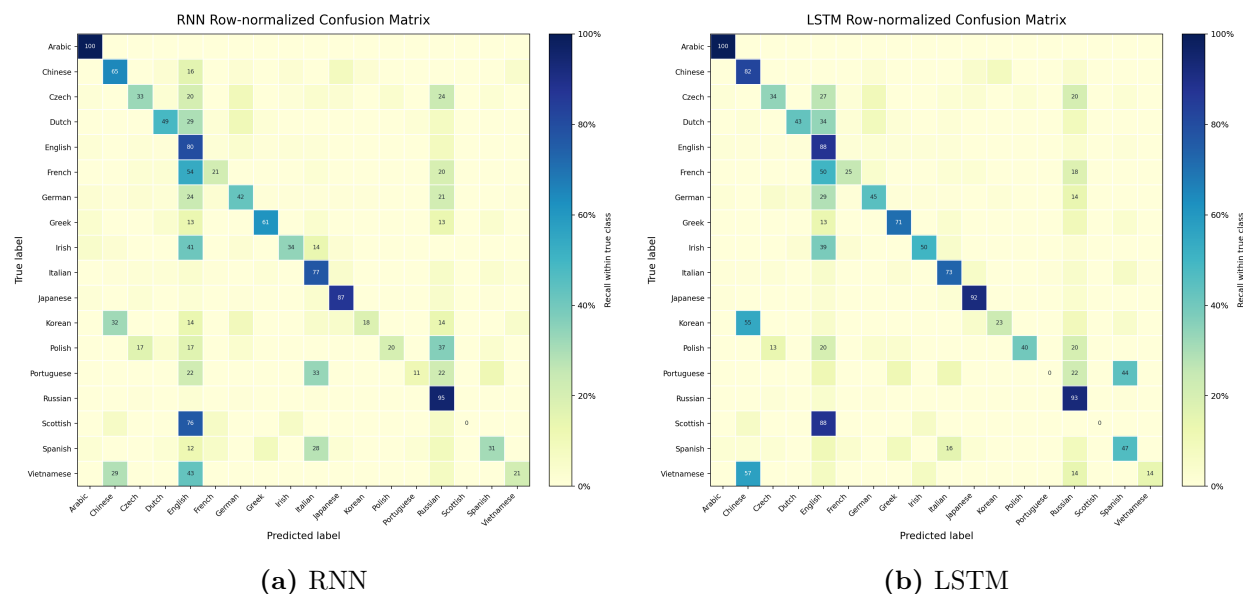


图 5: 姓名语言识别逐行归一化预测矩阵 (混淆矩阵, 单位: %)

存在相似后缀和拼写习惯。Scottish 在两个模型中几乎都被预测为 English, 说明该类别样本量和字符差异不足以支撑稳定边界; Portuguese、Vietnamese 也存在被吸收到 Spanish、Chinese 或 English 等类别的情况。相比之下, Arabic、Russian、Japanese 等类别的罗马化姓名模式更有区分度, 因此对角线比例较高。这说明字符级模型虽然无需人工规则, 但仍会受到数据分布和语言转写方式的影响。

本实验也显示, LSTM 并不会在所有类别上平均提升, 而是主要改善容易受长短期信息冲突影响的类别。对于短姓名, 普通 RNN 已能捕捉主要线索; 对于较长姓名或判别线索不在末尾的样本, LSTM 的记忆路径更有优势。

7 问题与改进方向

本实验的局限性主要在于数据类别不均衡和模型结构较简单。部分语言类别样本较少, 模型容易偏向高频类别; 同时本实验仅使用单向 RNN/LSTM, 没有利用后向字符信息。后续可以尝试 BiLSTM 或 GRU, 并加入类别加权损失、重采样或 focal loss, 以缓解类别不平衡。此外, 可以输出 Top-3 预测概率, 用于分析相似语言之间的置信度分布。

8 实验结论

本实验完成了 names 数据集上的字符级姓名语言识别。结果表明, 在相同数据、embedding 维度、隐藏层维度和优化器设置下, LSTM 以 83.44% 的验证准确率优于普通 RNN 的 81.97%。训练曲线和混淆矩阵进一步说明, LSTM 的门控细胞状态提升了序列信息保存能力, 使模型在相似语言类别之间的判断更稳定。通过本实验, 可以更直观地理解循环神经网络处理变长序列的方式, 以及 LSTM 相比普通 RNN 的实际优势。

参考文献

- [1] Jeffrey L. Elman. Finding Structure in Time. *Cognitive Science*, 14(2):179–211, 1990. https://onlinelibrary.wiley.com/doi/abs/10.1207/s15516709cog1402_1
- [2] Sepp Hochreiter and Jürgen Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9(8):1735–1780, 1997. <https://direct.mit.edu/neco/article/9/8/1735/6109/Long-Short-Term-Memory>
- [3] Klaus Greff, Rupesh Kumar Srivastava, Jan Koutník, Bas R. Steunebrink, and Jürgen Schmidhuber. LSTM: A Search Space Odyssey. *IEEE Transactions on Neural Networks and Learning Systems*, 28(10):2222–2232, 2017. <https://arxiv.org/abs/1503.04069>
- [4] Sean Robertson. NLP From Scratch: Classifying Names with a Character-Level RNN. PyTorch Tutorials, 2017. https://docs.pytorch.org/tutorials/intermediate/char_rnn_classification_tutorial.html
- [5] Adam Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *NeurIPS*, 2019. <https://arxiv.org/abs/1912.01703>